ASSIGNMENT 3 – TASK 7

DESIGN A MEMORY-MANAGEMENT PLAN FROM FIRST PRINCIPLES

Q7 - 10 Marks - Bloom's Level: K5 - Evaluate

1. Problem Interpretation and Design Objectives

The assignment specifies a smart-factory edge device with 2 GB of RAM and no swap disk. It runs a real-time control task that must never be delayed by a page fault, a machine-learning task requiring 1.5 GB, and a logging task. Further tasks may be added in the future. The design must protect the control task from page-fault delays while allowing the other work to use memory within the physical limit.

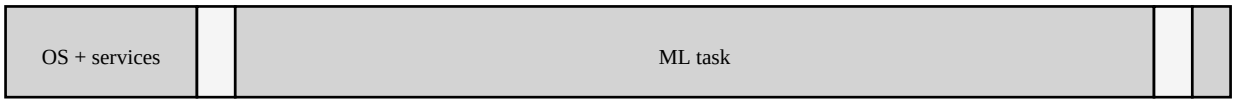
The question does not provide the control task's working-set size, logger's buffer size, operating-system reserve, or control-loop deadline. The following are explicit design assumptions used to turn the scenario into a measurable initial plan; they must be confirmed with the target hardware and application profile.

Task / area	Initial requirement and timing behaviour
Real-time control	Assume 64 MiB code/data/stack working set. Must be prefaulted and locked before the real-time phase. No major or minor page faults are permitted in its steady-state control loop. The required deadline D_RT must come from the control-system specification; the question does not state a numeric deadline.
Machine-learning	Needs 1,536 MiB (1.5 GiB) working set. Can tolerate demand paging and longer processing time, but its memory use must be bounded and monitored.
Logging	Assume a 64 MiB active ring buffer. Can tolerate buffered/asynchronous writes and non-real-time scheduling. Memory must be bounded so logging cannot consume the control reserve.
OS/kernel/services	Reserve 320 MiB for a minimal headless Linux image and required services. This is an assumption to validate at boot and peak load.
Safety headroom	Reserve 64 MiB for bursts and allocator/kernel variation; the initial budget is tight and must be stress-tested.

Measurable acceptance criteria: the real-time working set is locked and prefaulted before task activation; its measured steady-state loop has zero page faults; observed maximum loop delay is below the specified D_RT; ML memory is capped at 1,536 MiB; logger active memory is capped at 64 MiB; and the admission controller refuses new tasks if the remaining safety budget would be exceeded.

2. Stage 1 - Explicit 2 GiB Memory Budget

Illustrative 2 GiB memory budget (MiB)



320 + 64 + 1,536 + 64 + 64 = 2,048 MiB

Design assumptions; verify OS reserve and task working sets on the actual edge device.

Memory allocation	MiB	Reason
OS, kernel and services	320	Minimal edge Linux and required system services (assumption).
Real-time locked memory	64	Control code/data/stack working set (assumption).
Machine-learning task	1,536	1.5 GiB required by the question, represented as 1,536 MiB.

Memory allocation	MiB	Reason
Logging active buffer	64	Bounded ring buffer (assumption).
Safety/headroom	64	Short-lived bursts; small reserve requiring validation.
Total	2,048	Exactly 2 GiB.

This is a constrained plan: it uses the full 2 GiB budget and leaves only the stated 64 MiB safety margin. If measured OS, kernel, driver or workload use exceeds its allocation, the ML task must be reduced, offloaded, or the device RAM increased. Since the device has no swap disk, the plan must not rely on swap to absorb overload.

3. Stage 2 - Compare Candidate Memory-Management Mechanisms

Mechanism	Real-time control task	Machine-learning task	Logging task
Contiguous allocation	Can reserve a fixed physical/virtual region at startup; predictable but may be hard to grow and can fragment physical memory.	Large contiguous region is difficult to guarantee as memory grows; avoid requiring physical contiguity unless an accelerator/DMA device needs it.	A bounded contiguous ring buffer is simple, but fixed sizing may waste memory.
Paging	Use ordinary virtual pages but lock and prefault the control working set; page replacement must not evict it.	Suitable for flexible allocations; the ML task can tolerate demand faults within its memory cap.	Use paged buffers and bounded ring storage; page faults are acceptable outside the control loop.
Segmentation	Code, data and stack can be logically separated and protected, but segmentation alone does not prevent paging faults.	Useful as a logical code/data/heap/stack view, not a complete physical-memory policy.	Useful for logical organisation, but does not eliminate external fragmentation or page faults by itself.
Demand paging	Do not allow demand faults in the active control phase; prefault and lock the required pages before enabling the task.	Appropriate for loading model data as used, provided faults do not violate the ML task's latency requirements and memory pressure is monitored.	Suitable for non-critical log buffers and file-backed data; flush asynchronously.

Selected approach: Linux paging with an explicitly locked and prefaulted control working set, demand-paged ML memory, and a bounded logging ring buffer. `mlock()` prevents the specified region from being paged out. In a production build, code, stack and all loop data that must not fault should also be resident and prefaulted; `mlockall(MCL_CURRENT | MCL_FUTURE)` may be considered if the process's memlock limit and total memory budget permit it. The benchmark program uses `mlock()` on its controlled working region and prefaults it before measurement.

4. Stage 3 - Implementation: Locked and Unlocked Control-Loop Measurements

realtime_lock_bench.c creates an anonymous memory region and touches one byte per native page. In the unlocked case it calls MADV_DONTNEED between passes so the next pass must fault pages back in. In the locked case it calls `mlock()`, prefaults every page before timing, and then executes the same page-touch loop. `getrusage()` records minor and major page-fault deltas, and a monotonic clock records average and maximum observed per-touch latency.

The default benchmark working set is 4 MiB to fit common Linux memlock limits. It can be changed to a value up to 64 MiB after checking `ulimit -l`. If `mlock()` is denied, the program clearly marks the locked test as skipped; that run cannot be claimed as proof that locking worked.

Measured item	Unlocked (4 MiB)	Locked + prefaulted (4 MiB)
Number of measured page touches	3,072	3,072
Average touch time	1,491.0 ns	22.2 ns
Maximum observed touch time	134,722 ns	31 ns
Minor page faults during measurement	6,144	0
Major page faults during measurement	0	0

These are reference measurements from the artifact preparation environment (Debian GNU/Linux 13, GCC 14.2.0, Linux x86_64), not measurements from the student's own device. Re-run the script on the edge device and replace these values with its output. The observed maximum is a per-page-touch maximum for the measured run; it is not a formal hard-real-time worst-case execution-time guarantee.

Interpretation: the unlocked run shows page faults and a much larger observed maximum touch delay. The locked run reports zero page faults during the measured control-loop phase because its pages were locked and prefaulted first. This supports the design principle that the real-time task should not depend on demand paging while it is active.

5. Stage 3 - Page-Replacement Simulation for the Three Tasks

task_memory_sim.c models the proposed 2 GiB memory plan using 16 MiB abstract units. There are 128 model frames in total: 20 for OS/services, 4 pinned for the real-time task, 96 assigned to the ML task, 4 for the logging ring buffer, and 4 kept as headroom. The 16 MiB model unit is used only to keep the aggregate budget simulation compact; it is not the native Linux page size.

Task	Working set	Model frames	Simulated result
Real-time control	64 MiB	4 preloaded/pinned	0 runtime page faults after preloading
Machine-learning	1,536 MiB	96	96 initial demand faults; no replacement after warm-up for the model's 96-page active set
Logging	64 MiB	4	4 initial demand faults; no replacement after warm-up for the four-page ring buffer

The simulator prints PASS when the real-time phase has zero page faults. Its LRU implementation has O(NF) time for N references and F frames because each reference performs a linear search and a victim scan; space is O(F). The model gives each task its stated frame allocation, so it verifies the budget and residency assumption rather than modelling a fully general Linux global reclaim implementation.

6. Stage 3 - Verification and Interpretation

The measured benchmark verifies a no-page-fault locked phase in the test process, and the simulator verifies zero replacement faults for the prefaulted/pinned control pages. In the proposed device, the real-time task must be activated only after the lock operation and pre-touch have succeeded. The system should fail safe or refuse to enter the real-time state if the lock cannot be obtained.

Important limitation: zero page faults does not mean zero delay. `mlock()` removes page-out/page-in latency for the locked region, but scheduler pre-emption, interrupt handling, system-management events, bus contention, driver activity and non-resident code/stack can still delay a loop. The control system's deadline must be supplied by its engineering specification and verified with worst-case stress tests on the target device. A real-time Linux configuration, CPU affinity, real-time scheduling policy and bounded interrupt/driver paths may be needed for a strict guarantee.

7. Stage 4 - Adding a Fourth Task of 500 MiB

The current allocation is 2,048 MiB. A fourth task requiring 500 MiB raises the planned demand to 2,548 MiB, exceeding the 2 GiB device by 500 MiB before any additional headroom is included. The simulator therefore rejects the new task on the current device rather than stealing the real-time reserve or the ML task's required allocation.

Scenario	Planned memory	Decision
Three-task plan	2,048 MiB	Fits only under the stated assumptions; 64 MiB headroom.
Add 500 MiB fourth task	2,548 MiB	Reject on 2 GiB device; 500 MiB over budget.
Upgrade target	At least 3 GiB; 4 GiB preferred	4 GiB allows practical OS/driver growth and workload bursts.

Alternative actions are to move the new task to another edge node, reduce and revalidate a non-critical working set, or upgrade RAM. The real-time locked allocation must not be silently reduced to accommodate a new best-effort task.

8. Remaining Limitation

The principal limitation is that the real-time control and logging working-set sizes and deadline were not supplied. The 64 MiB control working set, 64 MiB logging buffer, 320 MiB OS reserve and 64 MiB headroom are assumptions. The benchmark uses a 4 MiB test area and a measured maximum from one run, while the memory simulator uses a coarse 16 MiB abstract unit. The design must be revalidated with the actual application traces, target kernel, drivers, memory pressure and required control-loop deadline.

9. Alternative Concept - RTOS with No Virtual Memory

If the edge device used a real-time operating system with no virtual memory, the design would use static physical memory partitions and fixed memory pools rather than demand paging and page replacement. The control task's code, stack and data would be allocated at build/startup time in a reserved region. There would be no VM page-in/page-out delay or page-replacement activity for the control task, so its memory-access path can have a more predictable bound.

Evaluation area	Linux with locked control memory	RTOS without virtual memory
Control-task delay	Locked/prefaulted pages remove page-fault delay for the protected working set. Remaining delay comes from scheduling, interrupts, drivers and hardware contention.	No demand paging or VM page faults. Worst-case delay is bounded by RTOS scheduling, interrupt latency, memory-bus and code-path WCET; usually more deterministic if the system is designed and measured correctly.
ML memory	1.5 GiB can be demand-paged/managed within a memory cap; page faults may slow ML, and no swap means pressure can lead to reclaim/OOM.	ML uses a fixed physical memory pool after RTOS/control/logging reservations. It can still receive 1.5 GiB under the illustrative budget, but memory cannot transparently grow beyond the assigned pool.
Adding tasks	Admission control and per-process virtual memory make deployment more flexible, but the physical budget still must fit.	New tasks require static memory allocation, partition/pool resizing, schedule analysis and often rebuild/revalidation; less flexible but deterministic.

Evaluation area	Linux with locked control memory	RTOS without virtual memory
Complexity	Virtual mapping and LRU/page replacement introduce memory-management overhead; the simulator is O(NF).	No runtime page replacement; fixed pool allocation can be O(1) with a bitmap/free list, but system integration and static budget analysis are more manual.

Under the same illustrative reservation, a 2 GiB RTOS device could assign 64 MiB to control, 1,536 MiB to ML, 64 MiB to logging and the remaining memory to the RTOS kernel and headroom. The exact ML capacity depends on the actual RTOS kernel, drivers and static allocations. Removing virtual memory improves determinism but sacrifices flexible process address spaces, demand paging and easier dynamic task addition.

10. Final Design Recommendation

Use a minimal Linux system with no swap, a 64 MiB real-time control working set locked and prefaulted before activation, a bounded 1.5 GiB ML memory allocation that can tolerate demand faults, and a 64 MiB asynchronous logging ring buffer. Reserve memory for the OS and retain a small safety margin. Use admission control to reject future tasks that exceed the physical budget. The benchmark and simulator support the principle, but final acceptance depends on reproducing the results on the actual edge hardware and meeting the control system's specified deadline.

If the control function has strict hard-real-time requirements that Linux cannot meet after full timing validation, an RTOS without virtual memory is the stronger alternative for the control plane. The ML task can run in a statically bounded pool or on a separate accelerator/node. A 500 MiB fourth task requires a memory upgrade; 4 GiB is the preferred practical target.

11. Execution Procedure and Evidence

1. Place realtime_lock_bench.c, task_memory_sim.c and run_task7.sh in one Task07 folder.
2. Open Ubuntu/WSL terminal in that folder.
3. Run chmod +x run_task7.sh.
4. Run ./run_task7.sh.
5. Review realtime_measurements.txt and confirm whether mlock succeeded.
6. Review task_memory_simulation.txt and verify the PASS line and fourth-task rejection.
7. Review environment.txt and record the compiler, Linux distribution, kernel and memlock limit used.
8. Capture screenshots of the benchmark and simulator outputs.

Evidence checklist: measurable requirements; 2 GiB memory budget; comparison of contiguous allocation, paging, segmentation and demand paging; mlock/prefault implementation; locked/unlocked observed delay and page-fault counts; three-task LRU simulation; zero control-task runtime faults; 500 MiB fourth-task admission decision; limitation; and RTOS alternative analysis.

Source alignment: Q7 - Design a Memory-Management Plan from First Principles, 10 Marks, K5 - Evaluate, page 30 of the supplied Assignment 3 question paper. Submission requirements are on page 31.